Evaluating Project Context Workflows for AI Assisted Software Development

Research-design white paper | 3 November 2026

Abstract. Project instructions and documentation can help coding agents locate relevant knowledge, but their practical value depends on the work needed to create, consume and maintain them. This paper defines a study of project-context workflows in Memos, an open-source application with existing agent-facing guidance. It asks what development benefits and resource costs these workflows produce, and what effort is required to keep context accurate and consistent as the code changes. The reference workflows are existing project practice and deliberately prepared context files. The comparison treats information differences as part of the workflow and records their resource implications. A shared evaluation flow connects pinned project and task inputs, isolated executions, independent correctness and regression checks, and phase-specific resource records. Maintenance observations connect source changes to context defects, repairs and observed human effort. The contribution is an evaluation design: no Memos execution results or organizational savings are reported.

1 Problem and motivation

A coding agent must work within a project's behavior, conventions and verification process. Source code supplies some of that information; instructions, package documentation and design records supply other parts. Maintaining extra guidance introduces a practical decision for an engineering organization: does the resulting workflow deliver enough useful development work to justify its resource use and upkeep?

That decision involves more than the number of successful patches. A workflow may complete the same tasks with fewer tokens, require more testing time, or shift work from repeated exploration into initial preparation. It may also create a second account of project behavior that becomes inaccurate as the source evolves. Measuring only the final agent session would hide these transfers of work.

The study therefore evaluates complete project-context workflows. Information quantity need not be equal across methods. A method that supplies additional facts is a legitimate practical comparison when its preparation, consumption and maintenance are visible. The resulting estimate concerns the whole workflow; it does not isolate the effect of document structure, information volume or a retrieval mechanism.

The paper follows a problem-to-design-to-limitations progression informed by the white paper of Esposito et al. (2024) and the research vision of Hassan et al. (2026). These works provide structural precedents for separating a proposal from its validation. Their visions are not evidence that the present context workflows improve coding performance.

2 Related evidence and limits

Mohsenimofidi et al. (2026) examined context-file adoption and content in open-source projects. Their MSR study reports varied organization, presentation styles and evolution, rather than an established common structure. It motivates testing how context characteristics affect agent outcomes and how context coevolves with code. As an observational study, it does not establish that a particular document organization improves task completion.

Chatlatanagulchai et al. (2026) characterize agent context files as evolving configuration-like artifacts in their TOSEM study, with prominent implementation, architecture and testing guidance. Their repository analysis strengthens the motivation to study maintenance as part of the intervention. It does not measure the causal return on context-authoring effort.

The experimental evidence also cautions against assuming that more guidance is beneficial. Gloaguen et al. (2026) compare repository-level context conditions on issue-resolution benchmarks. Their revised paper reports increased inference cost without a statistically significant task-success improvement relative to the no-file comparisons. The publication record identifies an ICLR workshop contribution; the later arXiv revision is the version considered here. Its findings concern the tested tasks, agents and context constructions, rather than every use of project knowledge.

Khatri (2026) compares no context, always-loaded instructions and selective wiki access across two agents on real repositories. The study, now published at the REALM workshop, reports no detectable correctness effect within its tested setting. In some repositories the

selective corpus includes more information, so delivery and content cannot be fully disentangled. Such differences can still be evaluated as complete workflows if resources are recorded. The paper offers an experimental starting point, not a universal equivalence result or a maintenance-cost estimate.

These sources supply descriptive evidence, experimental comparisons and research framing with different evidential roles. The cited context studies have been read for their methods and limitations; their released software and data have not been independently reproduced or audited for this study. This is a focused related-work selection, not a systematic literature review. The present design joins task outcomes and lifecycle resources within one project; it makes no claim that context-file comparisons are absent from the literature.

3 Questions and study setting

3.1 Two research questions

RQ1. What benefits and resource costs do project-context methods produce when coding agents perform development tasks?

RQ2. How much effort is required to keep that context accurate and consistent as the project changes?

RQ1 concerns task-level outcomes and resource consumption. RQ2 concerns the work and defects associated with maintaining the context supporting those tasks. They are related but require separate observations: a successful patch does not show that documentation is accurate, and an updated document does not show that an agent benefits from it.

3.2 Why Memos

Memos is an open-source, self-hosted note-taking application. Its inspected repository documentation describes a Go backend, a React/TypeScript frontend, and multiple database drivers. More directly relevant to this study, it maintains `AGENTS.md`, a domain terminology file, project documentation and a documented design-record lifecycle (Memos contributors, 2026).

The agent instructions cover repository layers, coding conventions, change routing and verification. They direct contributors to package-local documentation and ask that guidance be updated with relevant code changes. The documented commands include backend tests and frontend linting, unit testing and builds. These are inspected instructions, not checks executed in this study.

Memos provides a meaningful incremental baseline: the existing project has already invested in agent context. The study can ask whether additional preparation helps beyond that existing workflow, while recognizing that one application cannot represent all organizations or technology stacks.

Table 1: Reference workflows and required accounting.

Workflow	Context and accounting boundary
Existing project practice	Native instructions, documentation and ordinary code tools. Record automatic loading and observed discovery; retain existing context.
Prepared context files	Existing practice plus deliberately prepared, versioned project guidance. Record permitted sources, construction, review, exposure and creation/update resources.

The inspected documentation revision is commit `b40d8a7b573774c52b022c8d6537d6baf2129dc0`. It records the provenance of the selection evidence. It is not an experimentally validated snapshot: no local Memos setup, baseline suite or development task has been validated here. Documentation inspection and executable benchmark validation are distinct stages.

4 Reference workflows

Table 1 defines the reference workflows at the level needed for the research questions. Native project context remains available in both. The existing-workflow reference must not be relabelled as a no-context condition merely because no research-authored files are added.

For prepared files, reproducibility requires an inventory of source material, preparation procedure, resulting artifacts and delivery behavior. Preparation must be independent of hidden evaluation material and of other methods' outcomes. File hashes identify the actual treatment. A file that exists in a checkout is not necessarily loaded by an agent; availability and observed consumption must be recorded separately.

The comparison addresses the practical package of content and delivery. If a prepared workflow supplies more information, the inventory should make that difference explicit. It should also expose the cost of collecting, validating and updating the extra material. Equal information is unnecessary for this workflow question, although it would be needed for a later claim about representation alone.

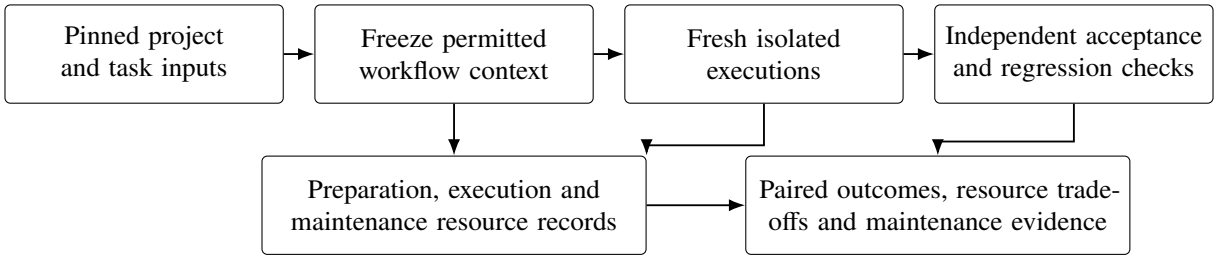


Figure 1: Shared evaluation flow. Executions use the same project state and task requirements, with workflow-specific context sealed in advance. No execution can read another workflow's outputs. Maintenance records concern shared source changes and each workflow's own permitted context history.

5 Shared technical evaluation flow

Figure 1 specifies the evaluation boundary. The orchestration connects immutable inputs to independent evidence. It does not prescribe the internal architecture of a project-knowledge method.

5.1 Pin and audit the inputs

An executable comparison requires a validated source revision, task specification, environment and evaluator. Pin these inputs and record their hashes or immutable identifiers. The task prompt and starting code state must be the same for paired comparisons. Preserve the original tests and audit the independent acceptance criteria before scored execution. Where a reference implementation is available, use it to check evaluator behavior without exposing it to context preparers or agents.

Record agent and tool versions, provider settings, context-loading behavior and permitted network access as part of the configuration. Preserve native instructions in the baseline inventory. Seal the method-specific context before task outcomes can influence its construction. A manifest should connect the common inputs, workflow context, configuration and execution identifiers.

5.2 Execute without cross-workflow learning

Each execution requires a clean checkout, a fresh conversation and separate writable state. Isolation includes agent memory, generated artifacts and caches that could contain task information. Shared dependencies may be read-only if they cannot carry another run's outputs. The research workspace, evaluator files, reference patches and other executions' logs must remain outside agent access.

Apply the same task requirements, objective evaluation, tool policy and execution-limit policy across workflows. Record execution order and any provider-side version limitations; randomization or counterbalancing can reduce order effects once the run plan is frozen. Public tests may guide implementation under the common policy. Hidden evaluation remains independent of the agent's development loop.

When an execution ends, stop the agent, preserve the final artifact and patch, and then run the independent evaluator. Record timeouts and infrastructure errors distinctly from implementation failures. Retries must retain their original attempts and resource records, so recovery does not erase the cost of an unsuccessful start.

5.3 Evaluate the submitted artifact

Acceptance checks ask whether the requested behavior is implemented. Regression checks ask whether previously supported behavior still works. Report these separately, alongside coverage limitations. Passing a suite is evidence about that suite; it is not proof that a change is production-ready or free of all defects.

Any human assessment requires a prespecified rubric, an identifiable review procedure and observed effort. Reviewers should be shielded from method labels where feasible. Narrative impressions from the agent are evidence of its process, not independent proof of task correctness.

6 Measurement and resource accounting

Table 2 connects the questions to evidence. All recorded quantities need an execution or maintenance-episode identifier and a source artifact. Missing measurements remain missing rather than becoming zeros.

6.1 Task and execution measures

Correctness reporting needs the number of evaluated tasks and attempts, not only a percentage. Retain paired task outcomes to show where methods agree or differ. Record unsuccessful submissions, stopping conditions, exceptions and evaluation failures. A timeout is a resource outcome even when it produces no passing patch.

Token records must preserve the service's accounting semantics. Adding cached input to an input total that

Table 2: Measurement dimensions and interpretation rules.

Dimension	Evidence to retain	Interpretation rule
Correctness	Frozen acceptance results, task criteria, submitted patch	Report tasks/attempts and denominators; separate partial results from full completion.
Regressions	Original checks and independent post-run results	Keep distinct from acceptance; describe coverage limits and unrelated baseline failures.
Tokens	Raw input, cached input, output and reasoning categories; provider definitions	Preserve overlaps. Cached input may be an input subset; reasoning may be included in output.
Time	Phase start/end, elapsed time, observed setup, queue and tool time	Separate agent execution, evaluation and preparation; elapsed time is not active human labor.
Preparation	Sources, procedure, artifact hashes/sizes, model usage and observed review	Expose initial costs and reuse scope; context size is not billed consumption.
Maintenance	Before/after source and context, inspected claims, defects, repairs and verification	Report audit scope, unresolved findings and observed update/review resources.
Human effort	Observed active time, role and activity for preparation, updating and review	Report only collected effort; do not infer staff savings from agent speed.

already includes it would inflate consumption; adding a reasoning subset to output would do the same. Keep raw events and document any derived total. File token counts describe the available context inventory and must not be substituted for actual model usage. Monetary costs require reliable billing data or dated rates mapped to the correct usage categories.

Time should be assigned to the activity that consumed it. Distinguish preparation, execution, evaluation and maintenance elapsed time. Record active human work separately when it is observed. Parallel activity and waiting can make these measures diverge. If a field is unavailable, report the missingness and its effect on interpretation.

6.2 Observe context maintenance

Maintenance requires code evolution rather than a single static snapshot. For a shared source change, record each workflow's prior context, the changed code and the context that remains afterward. Each workflow may retain its own intended history, but must not receive another method's findings. This preserves the distinction between a maintenance sequence and independent task executions.

An audit should link inspected context claims to source or test evidence. Classify supported statements, stale statements, contradictions, unsupported assertions and unresolved cases. Record the number and scope of claims inspected so that a larger document is not judged by a raw defect count alone. A historical design rationale is not automatically a claim about current behavior; its stated lifecycle matters.

Record how problems were detected, what was changed, who or what performed the update, the resulting artifact hash and how the repair was checked. Detection and repair latency require timestamps. Human maintenance effort requires observed active work. Editing more lines or generating a longer replacement document establishes neither greater labor nor greater accuracy.

6.3 Compare workflow resources

Keep initial preparation separate from recurring execution and update costs. If preparation is reused, state the reuse scope and allocation rather than concealing it inside a per-task figure. Record evaluation overhead separately so it is not mistaken for an agent's development time. Include failed attempts and retries in aggregate consumption.

Present correctness and resource use together. A faster unsuccessful attempt is not interchangeable with a correct implementation; a more expensive workflow may still produce more correct work. A resource-per-success measure needs an explicit numerator and denominator, includes failed-attempt resources, and is undefined if no task succeeds. These comparisons describe the observed workload and cannot supply a company-wide savings estimate.

7 Interpretation boundaries

The main unit of comparison is a paired task or shared maintenance episode. Repetitions help characterize execution variability but do not create independent new tasks. Analysis must preserve this grouping, report uncertainty

appropriate to the sample, and distinguish prespecified contrasts from exploratory explanations. A nonsignificant difference alone does not establish equivalence.

Information and delivery differences are deliberate parts of a workflow comparison. They limit component-level causal claims: an observed advantage cannot be assigned solely to file structure or a search mechanism. Additional controlled comparisons would be required for those explanations. Likewise, observed maintenance behavior must be distinguished from the mere existence of a documented maintenance policy.

One project, a particular task selection and a particular agent configuration support bounded conclusions. Public repositories may also appear in model training or expose prior solutions, so task provenance and contamination risks need explicit assessment. Independent tests reduce reliance on self-reports but cannot guarantee exhaustive behavioral or security coverage.

The design paper reports no Memos experiments. Its technical contribution is an auditable path from common inputs to independent outcomes, with context creation and upkeep inside the accounting boundary. The research can support an engineering decision only to the extent that correctness, consumed resources and observed effort are jointly measured for the workload actually studied.

References

Chatlatanagulchai, W., Li, H., Kashiwa, Y., Reid, B., Thonglek, K., Leelaprute, P., Rungsawang, A., Manaskasemsak, B., Adams, B., Hassan, A. E. and Iida, H. (2026), 'Agent READMEs: An empirical study of context files for agentic coding', *ACM Transactions on Software Engineering and Methodology* . Reviewed text: arXiv 2511.12884v2. https://doi.org/10.1145/3840295.

Esposito, M., Janes, A., Taibi, D. and Lenarduzzi, V. (2024), 'Generative AI in evidence-based software engineering: A white paper', arXiv white paper 2407.17440v3. Version of 22 August 2024. https://arxiv.org/abs/2407.17440v3.

Gloaguen, T., Mündler-Sasahara, N., Müller, M. N., Raychev, V. and Vechev, M. (2026), 'Evaluating AGENTS.md: Are repository-level context files helpful for coding agents?', arXiv 2602.11988v3, revised 29 September 2026. Earlier version: ICLR 2026 Workshop on Memory for LLM-Based Agentic Systems. https://arxiv.org/abs/2602.11988v3.

Hassan, A. E., Oliva, G. A., Lin, D., Chen, B. and Jiang, Z. M. (2026), 'Towards AI-native software engineering (SE 3.0): A vision and a challenge roadmap', *ACM Transactions on Software Engineering and Methodology* **35**(9), 1–25. Article 268. https://doi.org/10.1145/3807901.

Khatri, P. (2026), Do context files help coding agents? a two-agent ablation study on real repositories, *in* 'Proceedings of the Second Workshop for REsearch on Agent Language Models (REALM 2026)', Association for Computational Linguistics, pp. 57–67. Reviewed text: arXiv 2607.27250v1. https://aclanthology.org/2026.realm-1.5/.

Memos contributors (2026), 'Memos repository documentation', `README.md`, `AGENTS.md`, `CONTEXT.md` and `docs/design/README.md`. Commit `b40d8a7b573774c52b022c8d6537d6baf2129dc0`, inspected 3 November 2026. https://github.com/usememos/memos/tree/b40d8a7b573774c52b022c8d6537d6baf2129dc0.

Mohsenimofidi, S., Galster, M., Treude, C. and Baltes, S. (2026), Context engineering for AI agents in open-source software, *in* 'Proceedings of the 23rd International Conference on Mining Software Repositories (MSR 2026)', ACM. Reviewed text: arXiv 2510.21413v4. https://doi.org/10.1145/3793302.3793350.